

Pre-requisites

1. IBM Cloud Account Setup: [IBM Cloud Console](#)
2. Watsonx Access and Configuration: [Watsonx Overview](#)
3. Python Basics: [Python.org](#)
4. FastAPI Framework: [FastAPI Docs](#)

Project Workflow

1. Gemini AI Setup & Initialization :

- **Activity 1.1:** Set up Google Cloud access (or platform) with Gemini 1.5 Flash Pro API access.
- **Activity 1.2:** Configure API keys, usage limits, and test integration for multimodal support (text + image).
- **Activity 1.3:** Fine-tune prompts for budget interpretation and product recommendation generation.

2. Core Functionality Development :

- **Activity 2.1:** Design Flask-based backend structure for routing, session control, and planner modules.
- **Activity 2.2:** Create endpoints for Home Planner, Party Planner, and Jewelry Planner.
- **Activity 2.3:** Integrate Gemini prompts and templates for each planner with support for text + optional image inputs.

3. Backend Implementation (app.py) :

- **Activity 3.1:** Define Flask routes like `/generate-home`, `/generate-party`, `/generate-jewelry`.
- **Activity 3.2:** Organize backend using modular structure:
 - `routes/`: Endpoints
 - `services/`: Recommendation logic & AI calls
 - `models/`: Input/output schemas

- **Activity 3.3:** Setup CORS headers for frontend communication and implement basic session management.
- **Activity 3.4:** Include mock API calls or simulated scraping from Amazon, IKEA, Zomato, etc.

4. UI Development :

- **Activity 4.1:** Build responsive HTML forms for:
 - Home Interior Planner (room details, quantities)
 - Party Planner (event type, budget, guests)
 - Jewelry Planner (budget, occasion, outfit image upload)
- **Activity 4.2:** Add dynamic result display and formatting for recommendations using JavaScript.

5. Testing & Optimization :

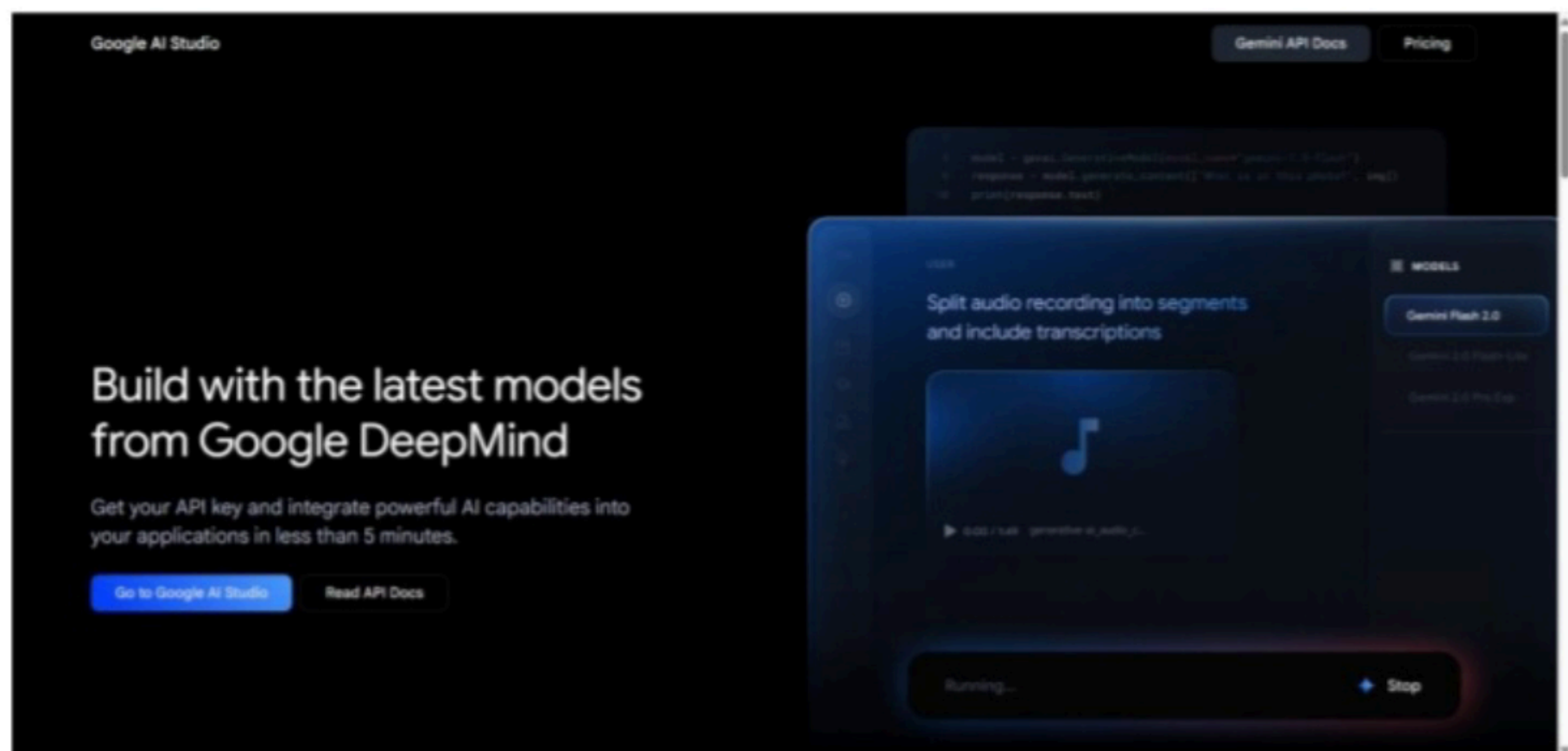
- **Activity 5.1:** Perform testing with real-world use cases across all three planners.
- **Activity 5.2:** Evaluate Gemini response quality, budget adherence, and platform accuracy.
- **Activity 5.3:** Optimize prompts and add validations to prevent input edge-case failures.
- **Activity 5.4:** Add fallback/default recommendations when AI returns insufficient results.

MILESTONE 1: Gemini AI Initialization :

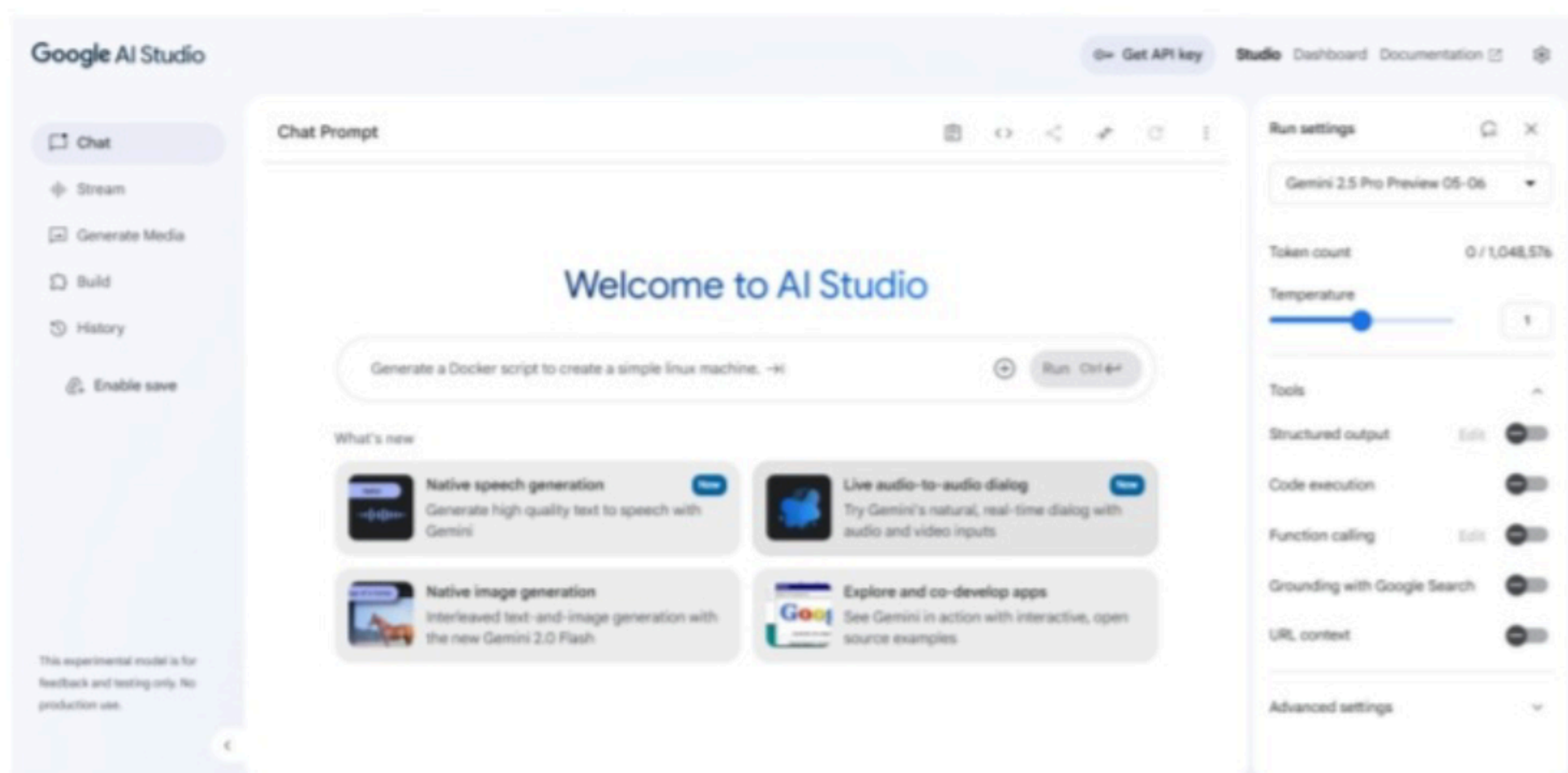
- **Objective:** Establish and configure the core AI infrastructure for PocketSmart AI.
- **Core Model:** Relies on Gemini 1.5 Flash Pro to interpret user inputs, process budgets, and generate personalized recommendations.
- **Key Capabilities:** Equips the development environment with authenticated access to Gemini's APIs for text-based reasoning and image analysis (specifically for the Jewelry Planner).

Activity 1.1: Set up Google Cloud account and enable Gemini API

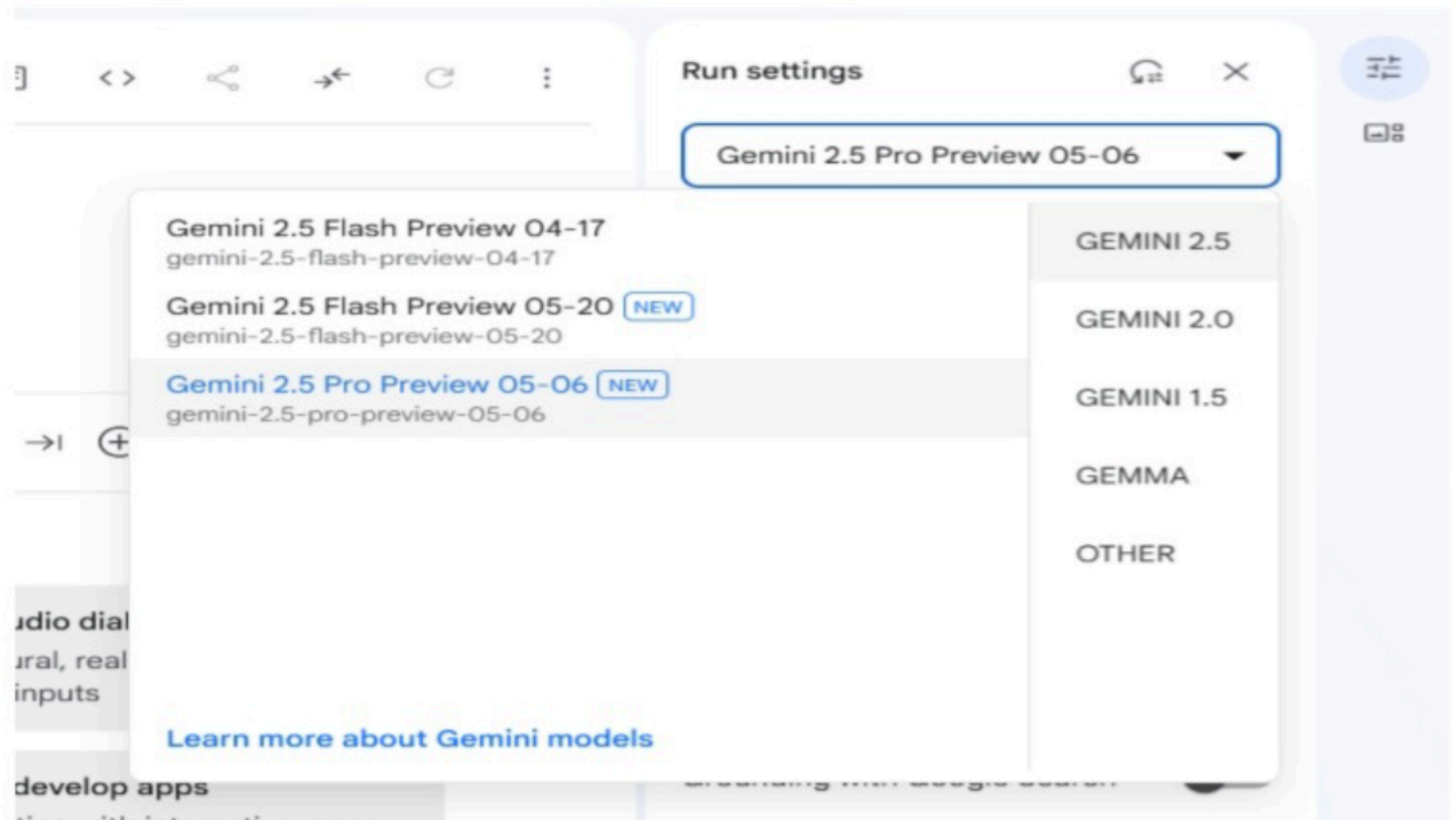
- Sign in or create a Google Cloud account via console.cloud.google.com.



- Accept all terms and conditions and click on Get API key



- You will get trending models here you can select the model here or you can set the model by go to the API Documentation.

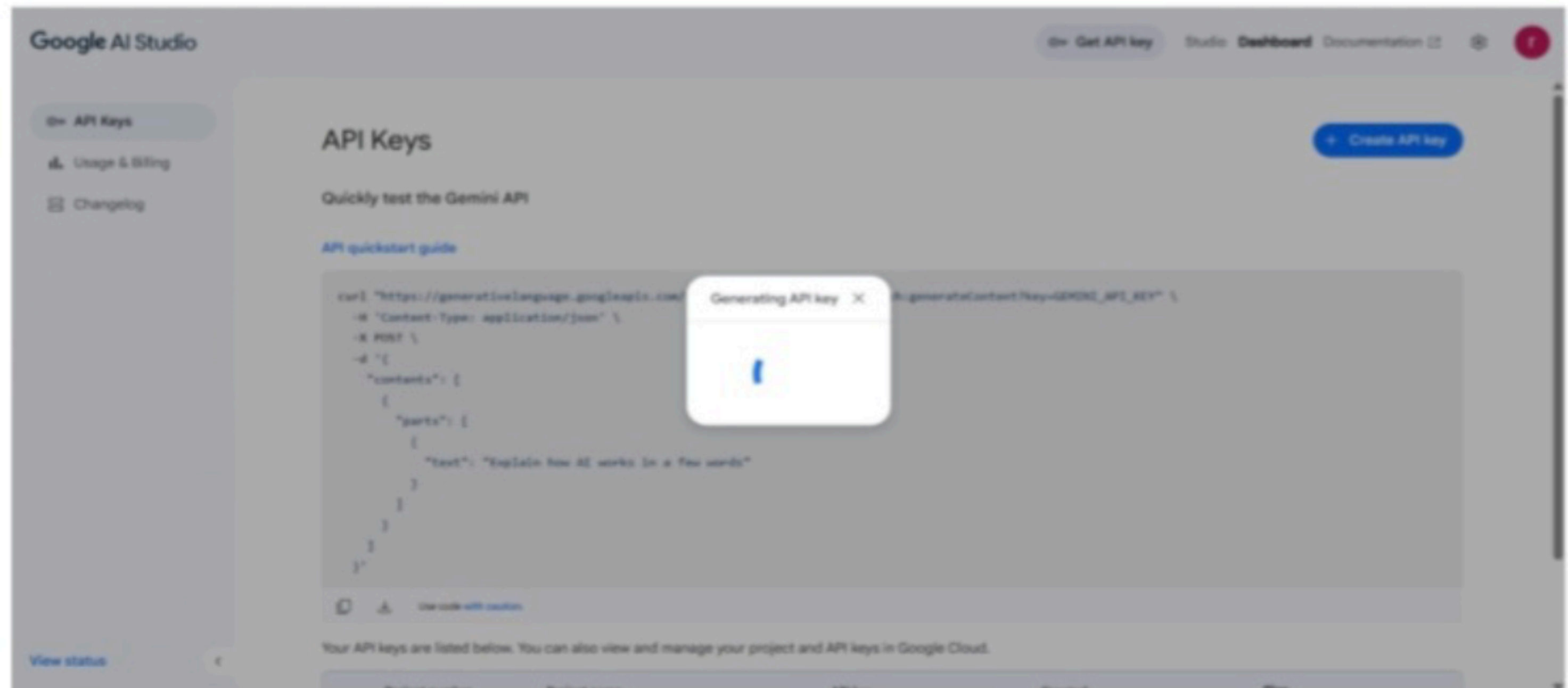


Activity 1.2: Configure access and permissions

- You will get interface like this and now click on Create API key



- Once click on API key you will the api key and make sure to copy the API key



Activity 1.3: Validate Gemini API connectivity

- Make a sample prompt call using Python to confirm integration.

```

app.py > ...
1  from google import genai
2  from google.genai import types
3  from PIL import Image
4  from io import BytesIO
5  import base64
6
7  client = genai.Client(api_key="AIzaSyDdzTxAlisLXLQ")
8
9  contents = ('Generate a detailed 2D architectural floor plan for a modern two-story house with a total area of 150 meters in length and 70 meters
10
11  response = client.models.generate_content(
12      model="gemini-2.0-flash-exp-image-generation",
13      contents=contents,
14      config=types.GenerateContentConfig(
15          response_modalities=['Text', 'Image']
16      )
17  )
18
19  for part in response.candidates[0].content.parts:
20      if part.text is not None:
21          print(part.text)
22      elif part.inline_data is not None:
23          image = Image.open(BytesIO((part.inline_data.data)))
24          image.save('gemini-native-image.png')
25          image.show()
  
```



- Test both text-only and image + text prompts to ensure multimodal compatibility.

